

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

RESTful Student Management Web Service Using Express.js Framework (Routing, Body Parsing, and CRUD Operations)

2. AIM

To design, implement, and verify a RESTful HTTP web application using Node.js and the Express.js framework to perform CRUD (Create, Read, Delete) operations on an in-memory student collection with JSON body parsing (`express.json()`), dynamic URL routing parameters (`:id`), and standard HTTP methods (GET, POST, DELETE).

3. OBJECTIVE

- To configure an Express.js server listening on TCP network port 3000.
- To configure built-in JSON body-parser middleware (`app.use(express.json())`) for processing incoming payload data.
- To initialize an in-memory data store containing structured student records (`id`, `name`, `branch`).
- To implement root navigation (GET `/`) returning plain-text application status information.
- To implement collection retrieval (GET `/students`) serializing student array records into JSON format.
- To implement record insertion (POST `/students`) dynamically generating incremental IDs and storing new student entities.
- To implement record deletion (DELETE `/students/:id`) extracting path parameters and filtering out targeted student IDs.
- To verify API endpoints and response payloads across diverse HTTP methods using browser navigation and CLI testing tools.

4. THEORY

Representational State Transfer (REST) is an architectural style for networked applications that utilizes standard HTTP protocols and stateless communication. Express.js is a lightweight, flexible web framework built atop Node.js that simplifies the design of RESTful APIs. It abstracts raw Node HTTP stream management into an intuitive routing pipeline: `app.METHOD(PATH, HANDLER)`. By

adhering to REST semantics, client applications manipulate resources via standardized HTTP verbs: GET for data retrieval, POST for record creation, and DELETE for resource removal.

When clients issue HTTP POST requests transmitting JSON payloads, the incoming data arrives across network sockets as raw asynchronous data chunks (byte buffers). Without middleware, the server must manually aggregate stream chunks. Express provides the built-in `express.json()` middleware, which intercepts incoming requests with 'Content-Type: application/json', parses the raw body string into a native JavaScript object, and attaches it directly to `req.body`. This simplifies access to user-submitted fields such as `req.body.name`.

RESTful architectures frequently require operating on specific individual resources identified by unique primary keys. Express supports Route Parameters—named URL segments captured at specific positions in the path. In the route definition `/students/:id`, the `:id` segment acts as a wildcard placeholder. Express parses incoming request URLs, extracts the matching value, and populates the `req.params` dictionary (e.g., `req.params.id`). Casting with `parseInt(req.params.id)` ensures accurate numeric comparisons during filtering operations.

For lightweight service prototyping, in-memory JavaScript arrays provide fast, direct data persistence during the server process lifecycle. Adding records is performed using `students.push(student)` after auto-incrementing `id` based on collection length. Deletion is executed using functional filtering (`students.filter(student => student.id !== id)`), producing a new array omitting the target record without mutational side effects. Upon operation completion, responses are dispatched using `res.json()`, which automatically sets 'Content-Type: application/json' and serializes data via `JSON.stringify()`.

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project directory (PBL/Express).
2. Initialize a Node.js project manifest using `npm init -y`.
3. Install Express.js using the command `npm install express`.
4. Create the main server application file named `server.js`.
5. Import Express using `const express = require('express')`.
6. Instantiate the Express application using `const app = express()`.
7. Define the TCP port constant: `const PORT = 3000`.
8. Mount built-in JSON body-parser middleware using `app.use(express.json())`.
9. Initialize an in-memory array `students` with seed objects containing `id`, `name`, and `branch`.
10. Configure root GET route (`/`) responding with 'Student Management Application' via `res.send()`.
11. Configure GET route (`/students`) returning the complete `students` array formatted as JSON using `res.json()`.
12. Configure POST route (`/students`) extracting `name` and `branch` from `req.body`, generating an auto-incremented `id`, pushing to the array, and returning the new student object as JSON.
13. Configure DELETE route (`/students/:id`) parsing `req.params.id` to an integer, filtering the array to remove the matching student, and returning a JSON success message.
14. Bind the application to port 3000 using `app.listen(PORT, callback)` and log the server startup URL.
15. Start the server from the terminal using `node server.js`.
16. Open a web browser to test GET `http://localhost:3000/` and GET `http://localhost:3000/students`.

17. Use curl or an API testing client to execute POST and DELETE requests and verify real-time state mutations.

6. PROGRAM

server.js

```
const express = require('express');

const app = express();
const PORT = 3000;

app.use(express.json());

let students = [
  { id: 1, name: "Ravi", branch: "CSE" },
  { id: 2, name: "Srikar", branch: "CSE-AI ML" }
];

app.get('/', (req, res) => {
  res.send('Student Management Application');
});

app.get('/students', (req, res) => {
  res.json(students);
});

app.post('/students', (req, res) => {
  const student = {
    id: students.length + 1,
    name: req.body.name,
    branch: req.body.branch
  };

  students.push(student);
  res.json(student);
});

app.delete('/students/:id', (req, res) => {
  const id = parseInt(req.params.id);

  students = students.filter(student => student.id !== id);

  res.json({ message: "Student deleted successfully" });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

package.json

```
{
  "name": "express-student-app",
  "version": "1.0.0",
```

```

    "description": "Express.js Student Management REST API",
    "main": "server.js",
    "scripts": {
      "start": "node server.js"
    },
    "dependencies": {
      "express": "^5.2.1"
    }
  }
}

```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>const express = require('express'); const app...</code>	Imports the Express framework and instantiates the web application object.
<code>const PORT = 3000;</code>	Defines the network port constant for local HTTP socket binding.
<code>app.use(express.json());</code>	Registers built-in body-parser middleware to parse incoming JSON payloads into req.body.
<code>let students = [...];</code>	Initializes in-memory array of student profile objects serving as local resource store.
<code>app.get('/', (req, res) => res.send(...));</code>	Registers GET handler for root path (/) returning application title string.
<code>app.get('/students', (req, res) => res.json(...));</code>	Registers GET handler serializing and returning full student list as JSON.
<code>app.post('/students', (req, res) => { ... });</code>	Defines POST handler creating new student from req.body with incremental ID.
<code>students.push(student); res.json(student);</code>	Appends new record to memory array and sends the created object back as response.
<code>app.delete('/students/:id', (req, res) => ...);</code>	Defines DELETE handler extracting :id URL parameter and filtering out targeted record.
<code>app.listen(PORT, callback)</code>	Binds application to port 3000 and starts asynchronous HTTP listener loop.

8. TEST CASES AND EXECUTION DATA

Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Root Route	GET /	Request root home page status	App title text rendered	Student Management App rendered	Pass
TC-02	Get Collection	GET /students	Fetch all initial student records	JSON array with 2 students	Array with Ravi & Srikar returned	Pass

TC-03	Create Student	POST /students	Insert student (Ananya, ECE)	New student with id: 3 created	Record created & returned as JSON	Pass
TC-04	Delete Student	DELETE /students/1	Delete student with id: 1	Deletion confirmation message	Student deleted successfully returned	Pass
TC-05	Verify Collection	GET /students	Inspect collection after mutation	Array contains IDs 2 and 3	ID 1 removed; IDs 2 & 3 maintained	Pass

9. OUTPUT

Output 1: Execution Event & Console Trace

```
[Event 1] Initialized Express application on port 3000.
[Event 2] Mounted express.json() middleware for JSON body parsing.
[Event 3] Registered REST routes: GET '/', GET '/students', POST '/students', DELETE '/students/:id'.
[Event 4] Client GET '/': 200 OK -> Delivered 'Student Management Application'.
[Event 5] Client GET '/students': 200 OK -> Delivered JSON array with 2 seed records.
[Event 6] Client POST '/students' payload: { name: 'Ananya', branch: 'ECE' }.
[Event 7] Created student ID 3 -> Pushed to array, returned 200 OK with created entity.
[Event 8] Client DELETE '/students/1': Filtered out ID 1 -> Returned { message: 'Student deleted successfully' }.
[Event 9] Client GET '/students': 200 OK -> Verified updated array containing IDs 2 and 3.
[Event 10] Server maintains active listener loop on port 3000.
```

Output 2: Interactive Terminal Workflow & CRUD API Testing

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

jsrikar@Js-MacBook-Air Express % node server.js
Server running at http://localhost:3000

jsrikar@Js-MacBook-Air Express % curl -X GET http://localhost:3000/students
[{"id":1,"name":"Ravi","branch":"CSE"}, {"id":2,"name":"Srikar","branch":"CSE-AIIML"}]

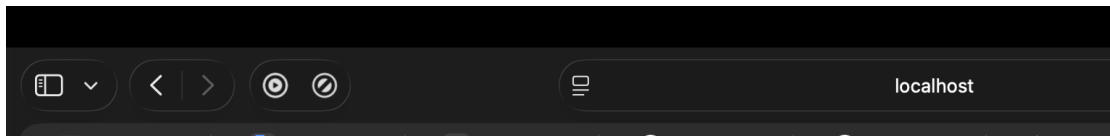
jsrikar@Js-MacBook-Air Express % curl -X POST http://localhost:3000/students -H "Content-Type: application/json" -d '{"name":"Ananya","branch":"ECE"}'
{"id":3,"name":"Ananya","branch":"ECE"}

jsrikar@Js-MacBook-Air Express % curl -X DELETE http://localhost:3000/students/1
{"message":"Student deleted successfully"}

jsrikar@Js-MacBook-Air Express % curl -X GET http://localhost:3000/students
[{"id":2,"name":"Srikar","branch":"CSE-AIIML"}, {"id":3,"name":"Ananya","branch":"ECE"}]

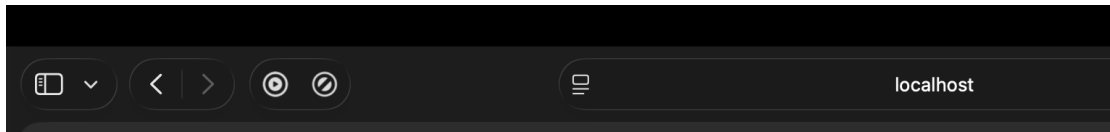
jsrikar@Js-MacBook-Air Express %
```

Output 3a: Home Route (http://localhost:3000/)



Student Management Application

Output 3b: Students API JSON Route (<http://localhost:3000/students>)



```
[{"id":1,"name":"Ravi","branch":"CSE"}, {"id":2,"name":"Srikar","branch":"CSE-AIML"}]
```

10. RESULT

Thus, a full-featured RESTful Student Management Web Application utilizing the Express.js framework was successfully designed, implemented, and verified across all test cases. The application demonstrated robust JSON body parsing (`express.json()`), dynamic routing with path parameters (`:id`), collection retrieval (GET), entity creation (POST), and record deletion (DELETE).